MATLAB - IMU Serial into MATLAB

Date: 22/11/2025
Tags:

Description

This documentation tracks the development of code to **assist** the firmware development for the ESP32 and MPU6050 IMU in Arduino IDE, using **MATLAB**.
This project roadmap will be divided in **categories** to separate different applications.
This **type** of documentation can be **categorized** as:

- **Algorithm Implementation**;
- **Read serial Data**;
- **Test**;
- **Etc**...
  Building on the calibrated output from V1.0 and V1.1, this implementation establishes a serial communication link to transmit sensor data directly into the MATLAB environment. This allows for **real-time data acquisition**, advanced visualization (plotting), and post-processing analysis that extends beyond the capabilities of the standard Arduino Serial Monitor.

Code

IMU Serial to MATLAB

```
clc
clear
close all;

% Inicialização do Filtro Complementar
alpha = 0.05; % Alpha entre 0 e 1 sendo muito pequeno
g = 9.81; % Aceleração gravítica igual a 9.81 m/s^2
roll_est_rps = 0; % Inicializar
pitch_est_rps = 0; % Inicializar
h = 0.333; % Tempo de amostragem

% 1. Configuration
portName = "COM9"; % Check your port
baudRate = 115200;
fileName = "imu_data_log.csv";

% 2. Connect to Serial
try
    s = serialport(portName, baudRate);
    configureTerminator(s, "CR/LF");
    flush(s);
catch ME
```

```
    error("Failed to connect. Is the Serial Monitor open in Arduino? Close it there first.");
end

% 3. Open the CSV file for writing ('w')
fileID = fopen(fileName, 'w');

% 4. *** CRITICAL STEP *** % Create a cleanup object. When this script ends (or you hit Ctrl+C),

% MATLAB will automatically execute the function inside this object.
cleanUpObj = onCleanup(@() cleanUpFunction(s, fileID));

% 5. Write the Header Row to the CSV
fprintf(fileID, "AcX,AcY,AcZ,GyX,GyY,GyZ\n");
fprintf("Logging started. Press Ctrl+C to stop.\n");

% 6. O Padrão de Regex (CORRIGIDO)
pattern = 'AcX\s*=\s*(-?[\d\.]+)\s+AcY\s*=\s*(-?[\d\.]+)\s+AcZ\s*=\s*(-?[\d\.]+)\s+GyX\s*=\s*(-?[\d\.]+)\s+GyY\s*=\s*(-?[\d\.]+)\s+GyZ\s*=\s*(-?[\d\.]+)';

% 7. O Loop
fprintf('A ler dados... Pressione Ctrl+C para parar.\n');

% Corrigir o erro de 'case' nas suas variáveis de inicialização
roll_est = 0; % Em vez de roll_est_rps
pitch_est = 0; % Em vez de pitch_est_rps

while(1)
    str = readline(s);
    tok = regexp(str, pattern, 'tokens');

    if isempty(tok)
    %fprintf('Skipped line: %s\n', str); % Descomente para depurar
        continue;
    end

    % data = [ax(m/s²), ay(m/s²), az(m/s²), gx(°/s), gy(°/s), gz(°/s)]
    data = str2double(tok{1});

    % --- 1. Extrair dados com as unidades corretas ---
    % Acelerómetro (m/s²)
    a_ms2 = [data(1), data(2), data(3)];

    % Giroscópio (graus/s)
    p_dps = data(4); % GyX
    q_dps = data(5); % GyY
    r_dps = data(6); % GyZ

    % Converter Giroscópio para Radianos/s (necessário para o filtro)
    p_rps = p_dps * (pi/180);
    q_rps = q_dps * (pi/180);
```

```
    r_rps = r_dps * (pi/180);

    % --- 2. Calcular Ângulos do Acelerómetro ---
    % Normalizar os valores m/s² de volta para 'g's dividindo por g
    % E usar 'clamp' (max/min) para evitar erros de asin()
    ax_g = max(min(a_ms2(1) / g, 1.0), -1.0);
    ay_g = a_ms2(2) / g;
    az_g = a_ms2(3) / g;

    % Usar atan2 é mais estável para o roll
    roll_est_acc = atan2(ay_g, az_g);
    pitch_est_acc = asin(ax_g);

    % --- 3. Aplicar Filtro Complementar Simples ---
    % As suas equações de 'roll_dot' estavam demasiado complexas e
    % misturavam variáveis. Para um filtro simples, basta isto:
    % Pitch (usa o giroscópio q, ou GyY)
    pitch_est = alpha * pitch_est_acc + (1-alpha) * (pitch_est + h * q_rps);

    % Roll (usa o giroscópio p, ou GyX)
    roll_est = alpha * roll_est_acc + (1-alpha) * (roll_est + h * p_rps);

    % (Nota: O Yaw não pode ser calculado sem um magnetómetro)
    % --- 4. Mostrar e Salvar ---
    % Mostrar os ângulos finais em Graus
    fprintf('Pitch (gr°) = %.2f | Roll (gr°) = %.2f\n', ...
    rad2deg(pitch_est), rad2deg(roll_est));

    % Salvar os dados FINAIS (floats) no ficheiro
    % Corrigido de %d para %.4f para salvar os decimais
    fprintf(fileID, "%.4f,%.4f,%.4f,%.4f,%.4f,%.4f\n", ...
    a_ms2(1), a_ms2(2), a_ms2(3), p_dps, q_dps, r_dps);
end

function cleanUpFunction(serialPort, filename)
    disp("Cleaning up...")

    try
        fclose(filename);
    catch ME
        error("File already closed");
    end

    try
        clear serialPort;
    catch ME
        error("Serial Port disconnected");
    end

    disp("Cleaning done.")
end
```

Code explanation

Complementary Filter Inicialization

```
alpha = 0.05;                   % Value of Alpha between 0 and 1
g = 9.81;                       % Gravitationl acceleration equal to 9.81
roll_est_rps = 0;               % Initialize the value for the filter
pitch_est_rps = 0;              % Initialize the value for the filter
h = 0.333                       % Sample time of the data
```

Configuration

```
portName = "COM9";              % The COM Port that Arduino is inserted
baudRate = 115200;              % Baudrate for serial communication
fileName = "imu_data_log.csv"   % File name in which will be saving data
```

- This **snippet** of code I have the constant values that are necessary for further applications in this code;

Connect to Serial Port

```
try
    s = serialport(portName, baudRate);
    configureTerminator(s, "CR/LF");
    flush(s);
catch ME
    error("Failed to connect. Is the Serial Monitor open in Arduino? Close it there first.");
end
```

- `serialport(port,baudrate)` This function connects to the **serial port** specified by **port** with a baud rate of **baudrate**.
- `try {statements}, catch exception {statments} and end` **Executes** the statements in the **try block** and **catches** the resulting **errors** in the **catch block**. **Exception** is an *MException* object that allows you to identify error. The **catch block** assigns the current **exception object** to the variable **exception**. This approach allows you to override the default error behaviour for a set of program statements, to see examples of this go to -> https://www.mathworks.com/help/matlab/ref/try.html?s_tid=srchtitle_support_results_1_try+catch.
- `configureTerminator(device, terminator)` Defines the terminator for both **read and write** communications with the specified serial port. Allowed terminator values are `"LF"` (default), `"CR"` , `"CR/LF"` , and integer values from 0 to 255. After you set the terminator, use `writeline` and `readline` to write and read ASCII terminated string data.
- `flush(device)` Flushes all data from both the input and output buffers of the specified serial port.
  `fileID = fopen(fileName,'w')` This line opens the file, specified by the variable `fileName` and with the `'w'` variable we are opening the file with the permission to **write**. More about this function in -> https://www.mathworks.com/help/matlab/ref/fopen.html?s_tid=srchtitle_support_results_1_fopen.
  `cleanUpObj = onCleanup(@() cleanUpFunction(s, fileID));` As the documentation says this line of code creates an object that, when destroyed, executes the function `cleanUpFunction` .

cleanUpFunction

```
function cleanUpFunction(serialPort, filename)
    disp("Cleaning up...")

    try
        fclose(filename);
    catch ME
        error("File already closed");
    end

    try
        clear serialPort;
    catch ME
        error("Serial Port disconnected");
    end

    disp("Cleaning done.")
end
```

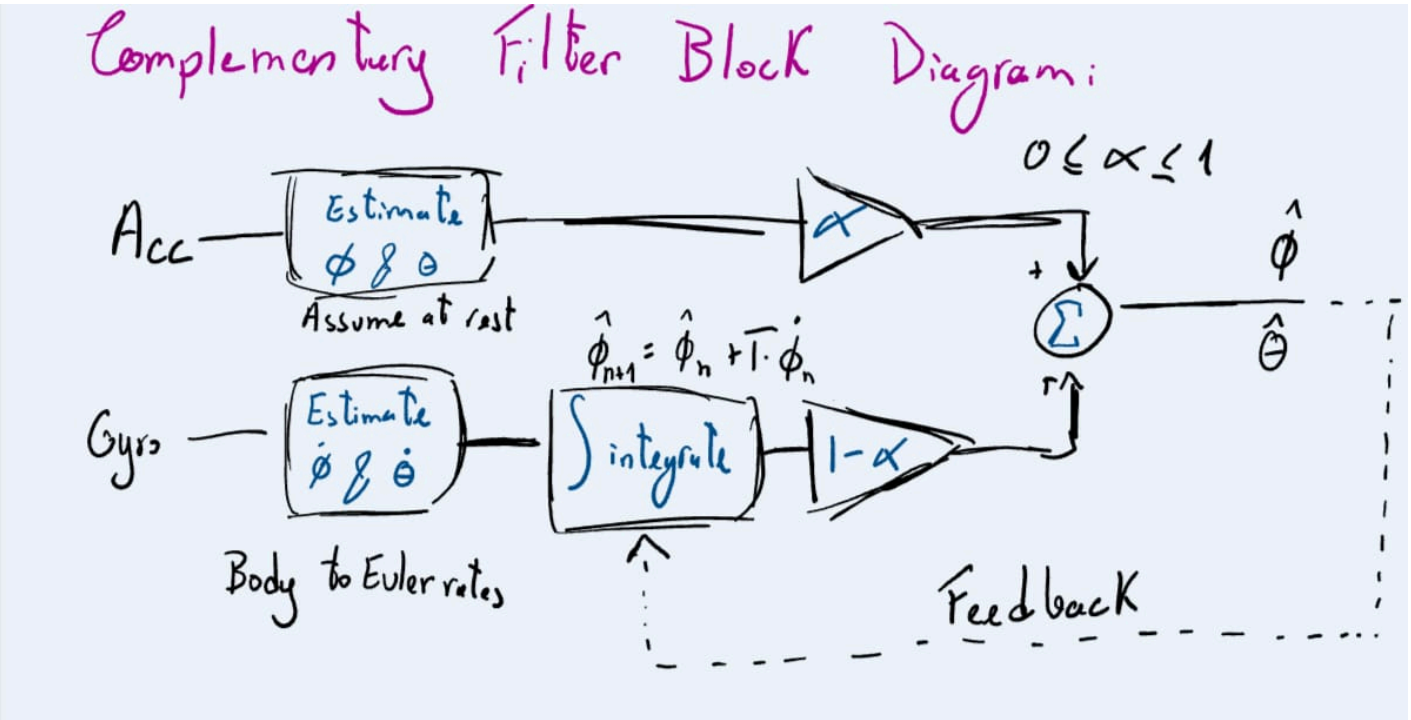
- This snippet of code is the `cleanUpFunction` .
´`fprintf(fileID, "AcX,AcY,AcZ,GyX,GyY,GyZ\n");` This line of code **prints** in the previous **file that was opened** the following string `"AcX,AcY,AcZ,GyX,GyY,GyZ\n"`

```
    pattern = 'AcX\s*=\s*(-?[\d\.]+)\s+AcY\s*=\s*(-?[\d\.]+)\s+AcZ\s*=\s*(-?[\d\.]+)\s+GyX\s*=\s*(-?[\d\.]+)\s+GyY\s*=\s*(-?[\d\.]+)\s+GyZ\s*=\s*(-?[\d\.]+)';
    (...)
    while(1)

    str = readline(s);
    tok = regexp(str, pattern, 'tokens');
```

- `pattern = (...)` `pattern` is used as the **expression parameter** of the `regexp` **function**. The **expression** can be specified as a character vector, a cell array of character vectors, or a string array. Each expression can contain characters, metacharacters, operators, tokens, and flags that specify patterns to match. For more information about the **expression parameter** -> https://www.mathworks.com/help/matlab/ref/regexp.html?s_tid=srchtitle_support_results_1_regexp
- `readline(s);` As said previously when explaining the `configureTerminator` **function** we use `readline` to read ASCII terminated string data from the specified serial device.
- `regexp(str, pattern, 'tokens');` Returns the output specified by `'tokens'` . This **outkey** specifies that the function has to return the text of each captured token in str. For more information about the function ->https://www.mathworks.com/help/matlab/ref/regexp.html?s_tid=srchtitle_support_results_1_regexp
`data = str2double(tok{1})` Transforms the data that was in **string** to **double**.
The rest of the code I will not document, but I will explained in descriptive text. My MATLAB will read all data that should appear in the Serial Monitor of the ArduinoIDE, then it will convert the data from the IMU to data with units (accelerometer -> m/s^2 and gyro -> rad/s). The the code will apply the necessary formulas to estimate the **roll & pitch** angle using the respective measurement unit model (**AT REST!!!!**) and then add both of the measurement units estimative. This way we

implement a complementary filter that can be described by the next block diagram:



Disclaimer

If I was not clear explaining the rest of the code please say it, I'm open to suggestions.
If the application of the filter is not clear to you or the normalize of units please contact me and I will anticipatedly share with you my notes about the series of videos that I'm taking about sensor fusion.